Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

TP 3 : Un protocole BLE pour transférer une image de la XIAO à la Portenta

Projet fédérateur EcoTri 4.0, sous-projet SP3 « Faire parler la station » et jalon J3

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

Objectifs du TP

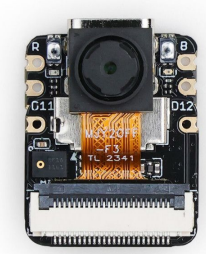
Le Tutorial 3 a fait de la Portenta un **serveur** GATT qui dialogue avec un téléphone. Dans ce TP, chaque équipe (3 ou 4 étudiants) met en œuvre le lien le plus exigeant de la station : la XIAO capture une vignette 96×96 et la transmet, **fragmentée et indexée**, à la Portenta qui la reconstruit, vérifie son intégrité et redemande les fragments perdus. Vous mesurerez ensuite le débit réel et le comparerez au modèle du cours. Le protocole documenté constitue le **jalon J3**.

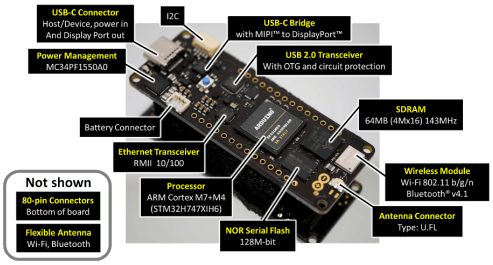
À la fin de ce TP, vous serez capable de :

- Spécifier un service GATT et un format de trames binaire, puis le faire implémenter par deux binômes de code différents (C++ et MicroPython).
- Programmer un serveur GATT sur l'ESP32-S3 (Arduino) qui notifie une image fragmentée.
- Programmer un client GATT sur la Portenta (`aioble`) : découverte, abonnement, reconstruction, CRC32, NACK sélectif.
- Mesurer débit, taux de perte et temps de transfert selon le MTU et le rythme d'émission, et les confronter au modèle théorique.
- Notifier au smartphone la décision issue de l'image reçue.

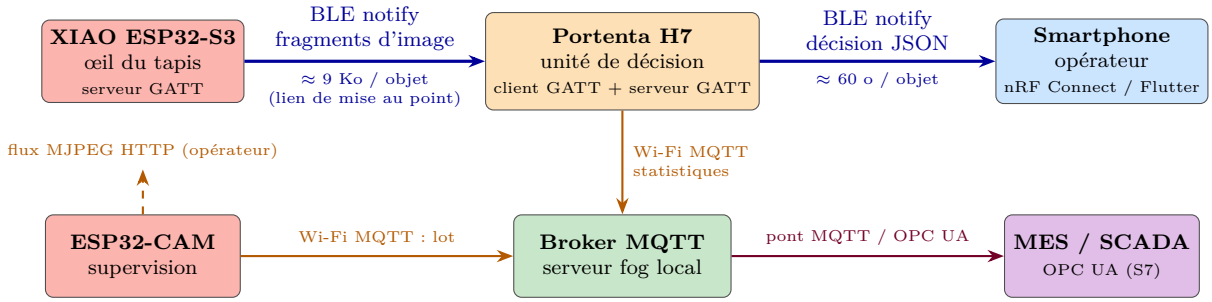
Livrable du jalon J3 (à déposer en fin de lecture) : spécification du protocole, codes, tableau et graphique des mesures (partie 6).

1 Contexte : le lien XIAO → Portenta





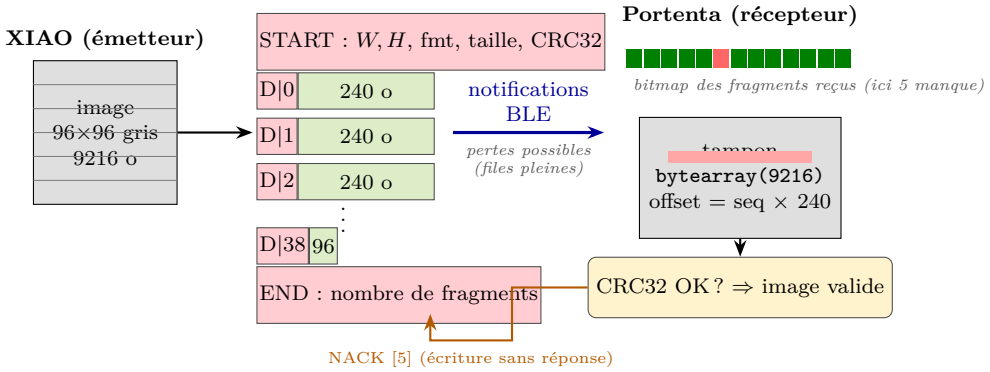
Les deux extrémités du lien : la XIAO ESP32-S3 Sense (« œil du tapis », serveur GATT) et la Portenta H7 (unité de décision, client GATT).



Règle de conception : plus on monte dans l'architecture, plus le message est petit et sémantique (pixels → étiquette → indicateur).

Le lien de ce TP dans l'architecture de la station : la flèche bleue de gauche (fragments d'image). La flèche de droite (décision vers le smartphone) est traitée en partie 5.

La XIAO est placée en amont, au-dessus du tapis ; elle filme chaque objet. Pour constituer le dataset de la lecture 5 et pour auditer les cas douteux, la Portenta doit recevoir la vignette complète. Le protocole retenu en cours est rappelé ci-dessous.



Le protocole à implémenter : trame START (métadonnées et CRC32), 39 trames DATA indexées, trame END ; le récepteur place chaque fragment à l'offset $seq \times P$, vérifie la bitmap et le CRC, puis envoie un NACK listant les fragments manquants.

2 Pré-requis

Matériel (par équipe)

- **XIAO ESP32-S3 Sense** avec sa carte caméra et son antenne, câble USB-C.
- **Portenta H7** + Vision Shield et antenne (Tutorial 3 terminé).
- Un smartphone avec **nRF Connect**.

Logiciel

1. **Arduino IDE 2.x**, paquet **esp32 by Espressif Systems** (version 3.x), carte **XIAO_ESP32S3**, option **PSRAM : OPI PSRAM** activée (TP1).
2. **OpenMV IDE** avec `aioble` fonctionnel (Tutorial 3, étape 1).
3. **Python 3** sur le PC avec `matplotlib` pour l'analyse des mesures.

3 Étapes détaillées du TP

Partie 1 : Spécifier avant de coder (15 min)

L'équipe se répartit en deux binômes : l'un code la XIAO (C++), l'autre la Portenta (MicroPython). Ils ne pourront se comprendre que si la spécification est **complète**. Recopiez et complétez le tableau suivant dans votre rapport ; générez votre propre base d'UUID (`uuidgen` sous Linux ou macOS, `[guid]::NewGuid()` sous PowerShell) pour ne pas brouiller les équipes voisines.

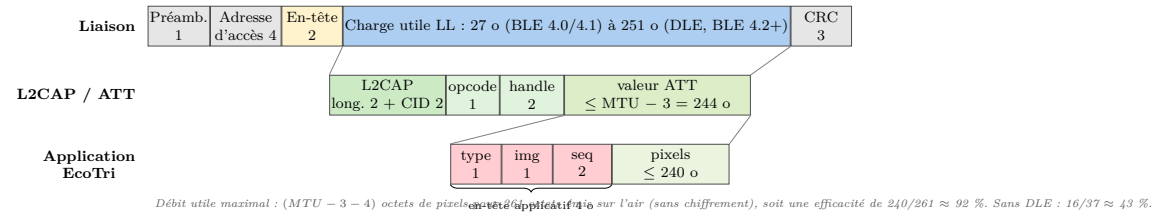
Élément	UUID (exemple)	Propriétés	Contenu
Service Image	`e0c00001-7b1a-4b2c-9d3e-ec0712a00000`	–	serveur : XIAO
IMG_DATA	`e0c00002-...`	notify	START, DATA, END
IMG_CTRL	`e0c00003-...`	write, write w/o resp.	commandes ci-dessous

Code	Commande	Paramètres (petit-boutiste)
0x10	CAPTURE	aucun : capturer et envoyer une nouvelle image
0x11	NACK	liste de numéros de fragments manquants (u16 chacun)
0x12	PAUSE	u8 : pause en ms entre deux notifications
0x13	ACK	u8 : identifiant de l'image reçue complète (la XIAO libère l'image)

Question

Vérifiez que les deux extrémités calculent le même CRC32 : sur la XIAO, `esp_crc32_le(0, (uint8_t*)"123456789", 9)` ; sur la Portenta, `binascii.crc32(b"123456789")`. Les deux doivent afficher `0xCBF43926`. Sinon, corrigez la convention (valeur initiale, inversion finale) **avant** d'aller plus loin et notez-la dans la spécification.

Partie 2 : Le serveur d'image sur la XIAO (35 min)



Une trame DATA : 4 octets d'en-tête applicatif et jusqu'à $MTU - 7$ octets de pixels dans la valeur ATT.

```
// TP3 - partie 2 : serveur GATT « Image » sur XIAO ESP32-S3 Sense (Arduino, esp32 3.x)
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLE2902.h>
#include "esp_camera.h"
#include "esp_crc.h"

#define SVC_IMG  "e0c00001-7b1a-4b2c-9d3e-ec0712a00000"   // à remplacer par votre base
#define CAR_DATA "e0c00002-7b1a-4b2c-9d3e-ec0712a00000"
#define CAR_CTRL "e0c00003-7b1a-4b2c-9d3e-ec0712a00000"
enum { T_START = 1, T_DATA = 2, T_END = 3 };

// Broches caméra de la XIAO ESP32-S3 Sense
#define PWDN_GPIO_NUM  -1
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM  10
#define SIOD_GPIO_NUM  40
#define SIOC_GPIO_NUM  39
#define Y9_GPIO_NUM    48
#define Y8_GPIO_NUM    11
#define Y7_GPIO_NUM    12
#define Y6_GPIO_NUM    14
#define Y5_GPIO_NUM    16
#define Y4_GPIO_NUM    18
#define Y3_GPIO_NUM    17
#define Y2_GPIO_NUM    15
#define VSYNC_GPIO_NUM 38
#define HREF_GPIO_NUM  47
#define PCLK_GPIO_NUM  13

BLECharacteristic *cData, *cCtrl;
volatile bool connecte = false, demande = false;
volatile uint16_t mtu = 23;
volatile uint8_t pauseMs = 3;
uint16_t nack[120]; volatile int nNack = 0;
camera_fb_t *fb = nullptr;          // image conservée jusqu'à l'ACK (pour les NACK)
uint8_t imgId = 0;
uint32_t nEnvoyees = 0;

class SrvCB : public BLEServerCallbacks {
  void onConnect(BLEServer *s) { connecte = true; Serial.println("# connecté"); }
  void onDisconnect(BLEServer *s) {
    connecte = false; mtu = 23; Serial.println("# déconnecté");
    BLEDevice::startAdvertising();
  }
  void onMtuChanged(BLEServer *s, esp_ble_gatts_cb_param_t *p) {
    mtu = p->mtu.mtu; Serial.printf("# MTU = %u\n", mtu);
```

```
  }
};

class CtrlCB : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *c) {
    auto v = c->getValue();                          // String (3.x) ou std::string (2.x
   )
    const uint8_t *d = (const uint8_t *)v.c_str(); size_t n = v.length();
    if (n == 0) return;
    switch (d[0]) {
      case 0x10: demande = true; break;
      case 0x11: { int k = 0;
        for (size_t i = 1; i + 1 < n && k < 120; i += 2) nack[k++] = d[i] | (d[i + 1]
    << 8);
        nNack = k; break; }
      case 0x12: if (n > 1) pauseMs = d[1]; break;
      case 0x13: if (fb) { esp_camera_fb_return(fb); fb = nullptr; } break;
    }
  }
};

void envoyer(uint8_t type, uint16_t seq, const uint8_t *p, size_t n) {
  static uint8_t buf[256];
  buf[0] = type; buf[1] = imgId; buf[2] = seq & 0xFF; buf[3] = seq >> 8;
  if (n) memcpy(buf + 4, p, n);
  cData->setValue(buf, n + 4);
  cData->notify();                                   // pas de confirmation applicative
  nEnvoyees++;
  if (pauseMs) delay(pauseMs);                       // contrôle de flux rudimentaire
}

size_t charge() { return mtu - 3 - 4; }              // octets de pixels par fragment

void envoyerFragment(uint16_t seq) {
  size_t P = charge(), off = (size_t)seq * P;
  if (!fb || off >= fb->len) return;
  envoyer(T_DATA, seq, fb->buf + off, min(P, fb->len - off));
}

void envoyerImage() {
  if (fb) esp_camera_fb_return(fb);
  fb = esp_camera_fb_get();
  if (!fb) { Serial.println("# capture échouée"); return; }
  imgId++;
  uint32_t crc = esp_crc32_le(0, fb->buf, fb->len);
  uint16_t N = (fb->len + charge() - 1) / charge();
  uint8_t meta[13];
  uint16_t w = fb->width, h = fb->height; uint8_t fmt = 0; uint32_t taille = fb->len;
  memcpy(meta, &w, 2); memcpy(meta + 2, &h, 2); meta[4] = fmt;
  memcpy(meta + 5, &taille, 4); memcpy(meta + 9, &crc, 4);
  uint32_t t0 = millis();
  envoyer(T_START, 0, meta, 13);
  for (uint16_t s = 0; s < N; s++) envoyerFragment(s);
  envoyer(T_END, N, (uint8_t *)&N, 2);
  Serial.printf("{\"img\":%u,\"mtu\":%u,\"pause\":%u,\"N\":%u,\"ms\":%lu,\"crc
    \":\"%08lx\"}\n",
                imgId, mtu, pauseMs, N, millis() - t0, (unsigned long)crc);
}

bool initCamera() {
```

```
  camera_config_t c = {};
  c.ledc_channel = LEDC_CHANNEL_0; c.ledc_timer = LEDC_TIMER_0;
  c.pin_d0 = Y2_GPIO_NUM; c.pin_d1 = Y3_GPIO_NUM; c.pin_d2 = Y4_GPIO_NUM; c.pin_d3 =
    Y5_GPIO_NUM;
  c.pin_d4 = Y6_GPIO_NUM; c.pin_d5 = Y7_GPIO_NUM; c.pin_d6 = Y8_GPIO_NUM; c.pin_d7 =
    Y9_GPIO_NUM;
  c.pin_xclk = XCLK_GPIO_NUM; c.pin_pclk = PCLK_GPIO_NUM; c.pin_vsync =
    VSYNC_GPIO_NUM;
  c.pin_href = HREF_GPIO_NUM; c.pin_sccb_sda = SIOD_GPIO_NUM; c.pin_sccb_scl =
    SIOC_GPIO_NUM;
  c.pin_pwdn = PWDN_GPIO_NUM; c.pin_reset = RESET_GPIO_NUM;
  c.xclk_freq_hz = 20000000;
  c.pixel_format = PIXFORMAT_GRAYSCALE;          // 1 octet par pixel
  c.frame_size = FRAMESIZE_96X96;                // 9216 octets
  c.fb_count = 2; c.fb_location = CAMERA_FB_IN_PSRAM; c.grab_mode =
    CAMERA_GRAB_LATEST;
  return esp_camera_init(&c) == ESP_OK;
}

void setup() {
  Serial.begin(115200); delay(1500);
  Serial.printf("# CRC test : %08lx (attendu cbf43926)\n",
                (unsigned long)esp_crc32_le(0, (const uint8_t *)"123456789", 9));
  if (!initCamera()) { Serial.println("# caméra KO"); while (true) delay(1000); }
  BLEDevice::init("EcoTri-XIAO");
  BLEDevice::setMTU(247);                        // MTU maximal proposé
  BLEServer *srv = BLEDevice::createServer();
  srv->setCallbacks(new SrvCB());
  BLEService *svc = srv->createService(SVC_IMG);
  cData = svc->createCharacteristic(CAR_DATA, BLECharacteristic::PROPERTY_NOTIFY);
  cData->addDescriptor(new BLE2902());           // CCCD : indispensable pour notify
  cCtrl = svc->createCharacteristic(CAR_CTRL, BLECharacteristic::PROPERTY_WRITE |
                                              BLECharacteristic::PROPERTY_WRITE_NR);
  cCtrl->setCallbacks(new CtrlCB());
  svc->start();
  BLEAdvertising *adv = BLEDevice::getAdvertising();
  adv->addServiceUUID(SVC_IMG);
  adv->setScanResponse(true);
  BLEDevice::startAdvertising();
  Serial.println("# publicité EcoTri-XIAO");
}

void loop() {
  if (connecte && demande) { demande = false; envoyerImage(); }
  if (connecte && nNack > 0) {
    int n = nNack; nNack = 0;
    Serial.printf("# NACK : %d fragments\n", n);
    for (int i = 0; i < n; i++) envoyerFragment(nack[i]);
    uint16_t N = fb ? (fb->len + charge() - 1) / charge() : 0;
    envoyer(T_END, N, (uint8_t *)&N, 2);
  }
  delay(2);
}
```

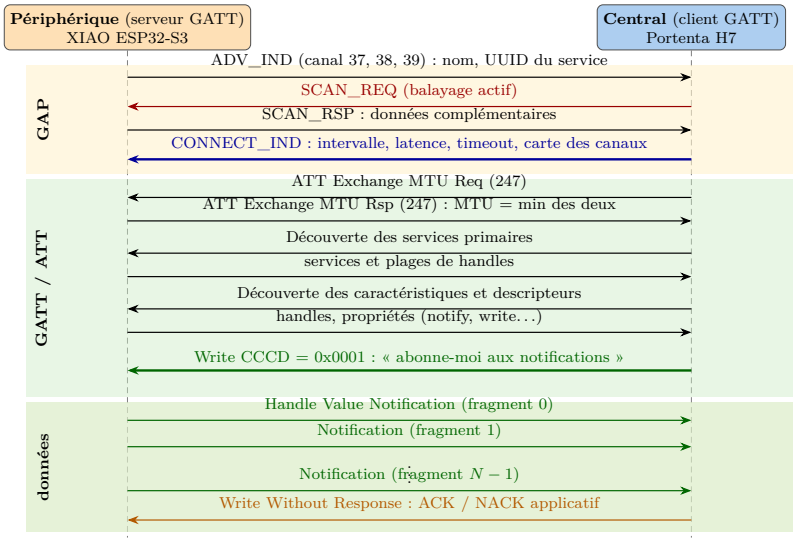
1. Téléversez le sketch et ouvrez le moniteur série (115200 bauds) : vérifiez la ligne `CRC test`.
2. Avec nRF Connect, connectez-vous à `EcoTri-XIAO`, abonnez-vous à IMG_DATA, puis écrivez `10` (type *BYTE ARRAY*) sur IMG_CTRL : une rafale de notifications doit apparaître dans le journal.

3. Notez la valeur du MTU affichée par la XIAO quand le téléphone se connecte.

Astuce

Si la compilation échoue sur `onMtuChanged`, votre version du paquet esp32 utilise une autre signature : supprimez la méthode et lisez le MTU avec `srv->getPeerMTU(srv->getConnId())` juste avant l'envoi. Si `esp_camera_init` échoue, vérifiez que la PSRAM OPI est activée dans le menu **Outils**.

Partie 3 : Le client de reconstruction sur la Portenta (35 min)



La séquence que le script de la Portenta doit dérouler : balayage, connexion, MTU, découverte, abonnement, réception, NACK.

```
# TP3 - partie 3 : client GATT sur la Portenta H7 (OpenMV, aioble)
import asyncio, aioble, bluetooth, struct, binascii, time, json

BASE = "-7b1a-4b2c-9d3e-ec0712a00000"          # même base que la XIAO
SVC_IMG  = bluetooth.UUID("e0c00001" + BASE)
CAR_DATA = bluetooth.UUID("e0c00002" + BASE)
CAR_CTRL = bluetooth.UUID("e0c00003" + BASE)
T_START, T_DATA, T_END = 1, 2, 3
MTU_TEST = 247                                  # 23, 100 ou 247 (partie 4)
PAUSE_MS = 3                                    # 0, 2, 5 ou 10 (partie 4)
N_IMAGES = 10
aioble.config(mtu=247)
print("CRC test : %08x (attendu cbf43926)" % (binascii.crc32(b"123456789") & 0
    xFFFFFFFF))

async def trouver_xiao():
    async with aioble.scan(10000, interval_us=30000, window_us=30000, active=True) as
     sc:
        async for r in sc:
            if r.name() == "EcoTri-XIAO" or SVC_IMG in list(r.services()):
                print("XIAO trouvée :", r.device, r.rssi, "dBm")
                return r.device
    return None
```

```
class Recepteur:
    def __init__(self):
        self.buf = None; self.recus = None; self.meta = None; self.img = -1; self.P =
     20

    def traiter(self, t):
        typ, img, seq = struct.unpack_from("<BBH", t)
        if typ == T_START:
            w, h, fmt, taille, crc = struct.unpack_from("<HHBII", t, 4)
            self.meta = (w, h, fmt, taille, crc); self.img = img
            self.buf = bytearray(taille); self.recus = set()
            return None
        if self.buf is None or img != self.img:
            return None                              # fragment d'une autre image :
    ignoré
        if typ == T_DATA:
            off = seq * self.P
            n = len(t) - 4
            self.buf[off:off + n] = t[4:]
            self.recus.add(seq)
            return None
        if typ == T_END:
            N = struct.unpack_from("<H", t, 4)[0]
            return [s for s in range(N) if s not in self.recus]   # liste des
    manquants

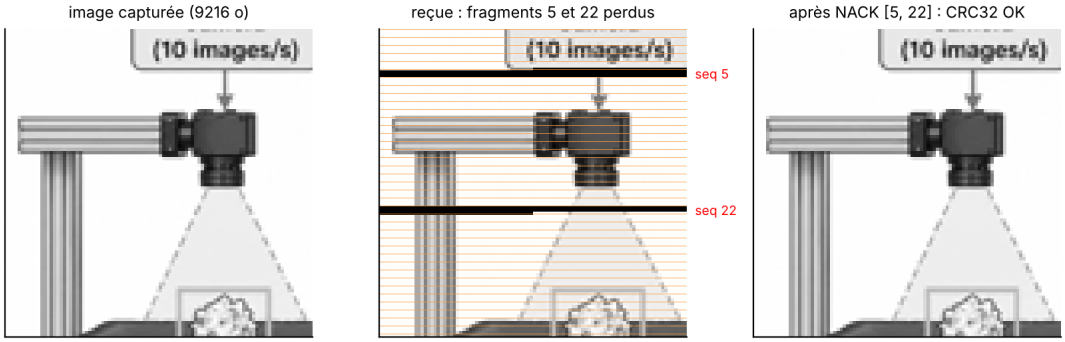
async def recevoir_image(data, ctrl, rx, mtu):
    rx.P = mtu - 7
    await ctrl.write(b"\x10", response=False)  # CAPTURE
    t0 = time.ticks_ms(); tours = 0; perdus_init = None
    while True:
        try:
            t = await data.notified(timeout_ms=3000)
        except asyncio.TimeoutError:
            print("# délai dépassé : image abandonnée"); return None
        manq = rx.traiter(bytes(t))
        if manq is None:
            continue
        if perdus_init is None:
            perdus_init = len(manq)
        if not manq:
            break
        tours += 1
        lot = manq[: (mtu - 4) // 2]           # une écriture tient dans le MTU
        await ctrl.write(b"\x11" + struct.pack("<%dH" % len(lot), *lot), response=
    False)
    dt = time.ticks_diff(time.ticks_ms(), t0)
    ok = (binascii.crc32(rx.buf) & 0xFFFFFFFF) == rx.meta[4]
    await ctrl.write(struct.pack("<BB", 0x13, rx.img), response=False)   # ACK
    w, h, fmt, taille, crc = rx.meta
    mesure = {"img": rx.img, "mtu": mtu, "pause": PAUSE_MS, "N": len(rx.recus),
              "perdus": perdus_init, "tours": tours, "ms": dt,
              "kbps": round(taille * 8 / dt, 1), "crc_ok": ok}
    print(json.dumps(mesure))
    return rx.buf if ok else None

def sauver_pgm(buf, w, h, nom):
    with open(nom, "wb") as f:                    # format PGM : lisible partout
        f.write(b"P5\n%d %d\n255\n" % (w, h)); f.write(buf)
```

```
async def central():
    dev = await trouver_xiao()
    if dev is None:
        print("XIAO introuvable"); return
    conn = await dev.connect(timeout_ms=5000)
    async with conn:
        mtu = await conn.exchange_mtu(MTU_TEST) if MTU_TEST > 23 else 23
        print("MTU =", mtu)
        svc = await conn.service(SVC_IMG)
        data = await svc.characteristic(CAR_DATA)
        ctrl = await svc.characteristic(CAR_CTRL)
        await data.subscribe(notify=True)        # écrit 0x0001 dans le CCCD
        await ctrl.write(struct.pack("<BB", 0x12, PAUSE_MS), response=False)
        rx = Recepteur()
        for k in range(N_IMAGES):
            buf = await recevoir_image(data, ctrl, rx, mtu)
            if buf is not None and k == 0:
                sauver_pgm(buf, rx.meta[0], rx.meta[1], "/rx_%d.pgm" % rx.img)
            await asyncio.sleep_ms(300)

asyncio.run(central())
```

1. Lancez le script (XIAO alimentée, **déconnectée** de nRF Connect : un périphérique n'accepte souvent qu'un central à la fois).
2. Ouvrez le fichier `rx_*.pgm` sur le disque USB de la Portenta (GIMP, IrfanView, ou `Image.open(...).show()` de la bibliothèque Pillow) : l'image est-elle correcte ?
3. Relevez les lignes JSON de mesure : nombre de fragments perdus au premier envoi, nombre de tours de NACK, durée.



Ce que produit une perte de fragments sans NACK (milieu) : des bandes noires de 2,5 lignes. Testez-le en commentant l'envoi du NACK, puis rétablissez-le.

Question

Provoquez des pertes : mettez `PAUSE_MS = 0` et ajoutez `await asyncio.sleep_ms(5)` dans la boucle de réception de la Portenta, comme si elle était occupée par une inférence. Combien de fragments sont perdus ? Où se produit la perte, puisque la couche liaison acquitte chaque paquet (indice : file de notifications côté client) ? Le NACK suffit-il à obtenir des images correctes ?

Astuce

Selon la version d'`aioble`, la file des notifications d'une caractéristique cliente est très courte (parfois une seule valeur) : toute notification arrivée avant l'appel suivant à `notified()` remplace la précédente. C'est un excellent cas d'école de l'argument de bout en bout : la radio n'a rien perdu, l'application si.

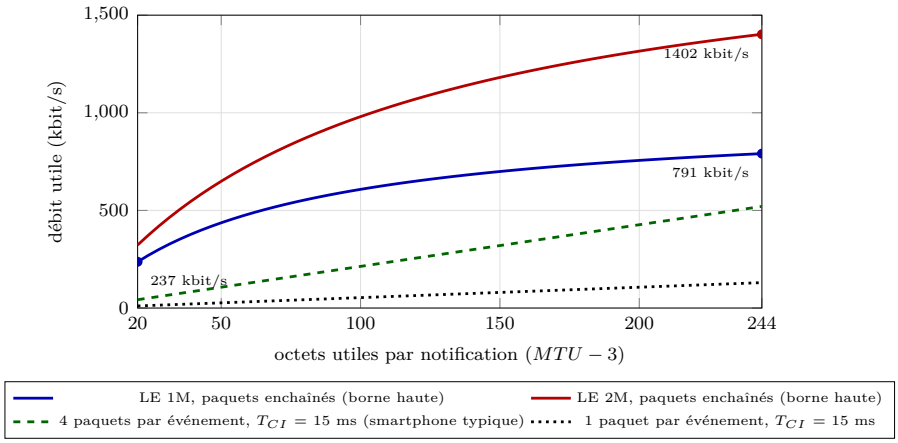
Partie 4 : Mesurer le débit et le confronter au modèle (25 min)

Faites varier deux paramètres : le MTU (`MTU_TEST` : 23, 100, 247) et la pause entre notifications (`PAUSE_MS` : 0, 2, 5, 10 ms). Pour chaque configuration, transférez 10 images et copiez les lignes JSON dans `mesures.jsonl`.

MTU	Pause (ms)	Fragments	Perdus (moy.)	Tours NACK	Durée (ms)	Débit (kbit/s)
23	3					
100	3					
247	0					
247	2					
247	5					
247	10					

Tableau à compléter : moyenne sur 10 images par configuration.

```
# analyse_debit.py - sur le PC : débit mesuré et borne théorique
import json, collections, matplotlib.pyplot as plt
data = [json.loads(l) for l in open("mesures.jsonl") if l.startswith("{")]
groupes = collections.defaultdict(list)
for d in data:
    if d["crc_ok"]: groupes[(d["mtu"], d["pause"])].append(d)
borne = lambda mtu: 8 * (mtu - 3) / (8 * (mtu + 4 + 10) + 380) * 1000   # kbit/s, LE 1M
for mtu in sorted({k[0] for k in groupes}):
    pauses = sorted(p for (m, p) in groupes if m == mtu)
    debits = [sum(d["kbps"] for d in groupes[(mtu, p)]) / len(groupes[(mtu, p)]) for p in pauses]
    plt.plot(pauses, debits, "o-", label="MTU %d (borne %.0f kbit/s)" % (mtu, borne(mtu)))
plt.xlabel("pause entre notifications (ms)"); plt.ylabel("débit utile mesuré (kbit/s)")
plt.grid(alpha=0.3); plt.legend(); plt.title("Transfert d'image XIAO -> Portenta (J3)")
plt.savefig("debit_J3.png", dpi=150)
```



Bornes théoriques du cours, pour comparaison avec vos mesures.

Question

1. Quelle part de la borne théorique atteignez-vous ? Avec une pause p et un envoi de durée négligeable, le débit ne peut dépasser $8\,(MTU - 7)/p$: vos mesures suivent-elles cette loi ?
2. Pourquoi le débit à pause nulle n'est-il pas forcément le meilleur ? Reliez votre réponse au nombre de fragments perdus et aux tours de NACK.
3. Le transfert d'une image tient-il dans l'échéance de 600 ms de la station ? Que concluez-vous sur l'usage de ce lien en production (lien de mise au point ou lien de décision) ?

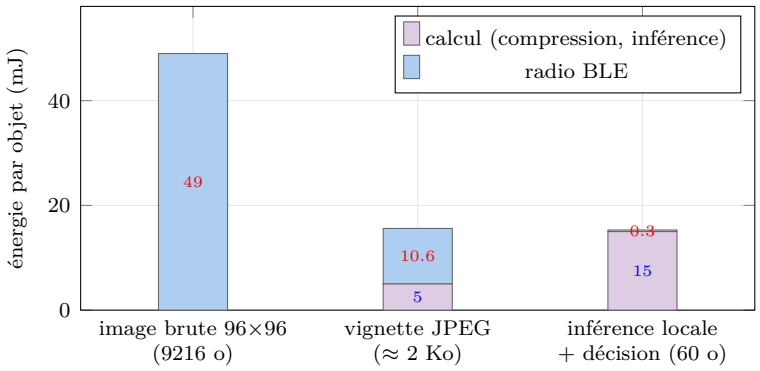
Partie 5 : Notifier la décision au smartphone (15 min)

La Portenta peut être **centrale** face à la XIAO et **périphérique** face au smartphone en même temps. Réutilisez le service Décision du Tutorial 3 : à chaque image valide, calculez une décision (pour l'instant, la luminance moyenne ; demain, le classifieur) et notifiez-la.

```
# A ajouter : service Décision du Tutorial 3 (SVC_DEC, decision, periph()) puis
def decider(buf):
    L = sum(buf[::16]) // len(buf[::16])        # luminance moyenne échantillonnée
    cls = "metal" if L > 150 else ("carton" if L > 90 else "vide")
    return {"v": 1, "src": "xiao", "cls": cls, "L": L, "t": time.ticks_ms()}

# dans central(), après une image valide :
#     decision.write(json.dumps(decider(buf)).encode(), send_update=True)

async def main():
    await asyncio.gather(periph(), central())   # deux rôles GAP simultanés
asyncio.run(main())
```



Le principe illustré par cette partie : vers le smartphone, on envoie 60 octets de décision au lieu de 9216 octets d'image.

Question

Mesurez avec nRF Connect le délai entre deux décisions reçues. Si votre firmware refuse les deux rôles simultanés, alternez-les (périphérique entre deux transferts) et mesurez le coût de cette alternance. Quel principe du cours cette partie illustre-t-elle ?

Partie 6 : Livrable du jalon J3 (15 min)

a) Spécification du protocole (une page au plus) : table GATT complète (UUID, propriétés, rôles), format de chaque trame et de chaque commande avec un exemple en hexadécimal, convention CRC32 et ordre des octets, format JSON de la décision (champs, types, unités, version), comportement (délais d'expiration, NACK, reprise après déconnexion).

b) Tableau des mesures

Mesure	Valeur	Conditions
MTU négocié XIAO / Portenta		
Débit utile maximal mesuré (kbit/s)		MTU, pause
Borne théorique correspondante (kbit/s)		modèle du cours
Temps de transfert d'une image 96×96 (ms)		meilleure configuration
Taux de perte au premier envoi (%)		pause nulle
Taux d'images valides (CRC OK) après NACK		sur 10 images
Délai entre deux décisions sur le smartphone (ms)		partie 5

c) Graphique `debit_J3.png` commenté : écart à la borne et goulot identifié (radio, pile, MicroPython).

d) Analyse : en dix lignes, justifiez le choix du lien et du format pour chaque flux de la station (image de mise au point, décision, statistiques) en citant vos mesures.

4 Grille d'évaluation du jalon J3

Critère	Attendu	Points
Spécification	complète et non ambiguë : une autre équipe pourrait l'implémenter sans question	5
Fonctionnement	image reconstruite, CRC vérifié, NACK opérationnel, décision notifiée au smartphone	5
Mesures	plan d'expérience suivi, moyennes sur 10 images, conditions précisées	4
Analyse	comparaison au modèle, goulot identifié, choix des liens argumentés	4
Qualité	code commenté, constantes partagées, unités et ordres de grandeur	2
Total		**20**

5 Ressources

— OpenMV, module `aioble` (rôle central, abonnement, `notified`) : docs.openmv.io.

— Espressif, bibliothèque BLE du paquet Arduino-ESP32 et exemple *BLE_notify* : github.com/espressif/arduino-esp32.

— Seeed Studio, *XIAO ESP32S3 Sense : caméra et Bluetooth* (wiki.seeedstudio.com).

— Bluetooth SIG, *Core Specification 5.4*, vol. 3, partie F (ATT) et partie G (GATT).

— A. S. Tanenbaum, N. Feamster, D. Wetherall, *Computer Networks*, 6e éd., chapitre 3 (détection d'erreurs, ARQ, contrôle de flux).

— J. H. Saltzer, D. P. Reed, D. D. Clark (MIT), « End-to-End Arguments in System Design », *ACM TOCS*, 1984.